

Correspondence

Using Multi-Agent Simulation and Learning to Design New Business Processes

Fu-ren Lin and Yu-Hua Pai

Abstract—Business process modeling, analysis, and then redesign are the central task in the efforts of reengineering business processes. Frequently reviewing and promptly changing business processes to adapt to new business environment is the key to maintain agility under the competitive global market. It is imperative to enhance the adaptability of business processes. This paper proposes a multi-agent information system (MAIS) based on Swarm, a multi-agent simulation platform, to simulate business processes, and incorporate reinforcement learning to obtain better process adaptability. The resulting system, called BPSLS, is elaborated and evaluated by the order fulfillment process in supply chain networks.

Index Terms—Business process reengineering, multi-agent system, reinforcement learning, Swarm.

I. INTRODUCTION

The efforts to capture and analyze existing business processes, and then to redesign new processes are emphasized by business process reengineering methodologies. Several methods for modeling business processes have been proposed in literature and practical usage, but few can handle process modeling, analysis, and design altogether. Business environment is changing rapidly due to the innovation of technology and global competition, which leads business processes to short life cycles. Frequently reviewing and promptly changing business processes to adapt to new business environment is the key to maintain agility under the competitive global market. Therefore, it is imperative that a process modeling method should possess such capabilities as process representation, analysis, and design, so that a better process design can be generated and evaluated for different business environments.

A business process can be described in five elements: 1) a business process has its customers, 2) a business process is composed of activities, 3) these activities are aimed at creating value for the customers, 4) activities are operated by actors which may be humans or machines, and 5) a business process often involves several organizational units which are responsible for a whole process [1], [2]. A process modeling method should be able to represent such concepts as activity, actor, organization, etc., so that the essential process characteristics can be captured. This article utilizes a multi-agent information system (MAIS), a multi-agent simulation platform, to model business processes [3]. We view a business process as a multi-agent information system, where agents are actors performing certain tasks according to organizational roles. Agents communicate and coordinate based on the information infrastructure in achieving process goals. The agent, task, organization, and information infrastructure, the four constructs of the MAIS, are implemented in the Swarm simulation system.

The Swarm simulation system is implemented to simulate business process, so that through experiments, we can evaluate process performance under different circumstances. However, how to adapt to the

changes in business environments by (re)designing (new) processes effectively and promptly relies on continuous learning and improving. A reinforcement learning approach, adopted from human's cognition to interact with its environment, is embedded in agents in order to facilitate (new) process design.

This paper elaborates the strengths of the multi-agent information system on simulating, analyzing, and learning business processes, which achieve better process agility. We describe the MAIS for process simulation and analysis, and then construct the Swarm with reinforcement learning capability, called BPSLS, for illustrating the MAIS application on designing a new business process. Finally, we select an order fulfillment process in a supply chain to show the BPSLS performance.

II. BUSINESS PROCESS MODELING AND ANALYSIS

Several BPR methodologies are proposed to guide the process reengineering procedure [1], [5], [8]. Some activities among various process reengineering methods are common, such as *existing process representation and diagnosis, and then target process redesign*. These activities rely on the central tasks: *process modeling, analysis, and then redesign*.

Inspired by the development of distribution artificial intelligence (DAI) on the multi-agent systems (MAS), and the characteristics of business processes, a MAIS is proposed to model the order fulfillment process in supply chain networks [3]. The MAIS consists of four components: *agents, tasks, organizations, and information infrastructure*. The four components are described as follows:

- 1) An *agent* is an active object, which possesses certain capabilities to perform tasks, and communicates with other agents based on the organizational structure.
- 2) Tasks are things which agents are assigned to work on. Depending on the structure of the agents in an organization, tasks can be *decomposed* by an agent and distributed to other agents, or tasks accomplished by agents are *composed* as the global accomplishment.
- 3) An *organization* within a MAIS defines the relations between agents. Agents in an organization interact through control and information linkages between them.
- 4) *Information infrastructure* facilitates the communications between agents. Information is shared either by common accessible databases or through message passing.

Agents in the MAIS are distributed geographically, are autonomous but interdependent in their task environments, are embedded with self-organization capability, and are adaptive to changes in their environment. A simulation system should be able to model these features in an efficient and effective way, so that the interaction between agents can be represented, and the system dynamics can be identified. Swarm is a multi-agent simulation software platform for the study of complex adaptive systems. A prototype of the Swarm simulation system was developed at the Santa Fe Institute and aimed to provide a general purpose simulation tool for building simulation models [6]. Table I describes the mapping of properties between the MAIS and Swarm.

III. THE REINFORCEMENT LEARNING

The reinforcement learning (RL) concept is based on the following observation: A learner continuously receives sensory inputs from the

Manuscript received October 7, 1998; revised February 17, 2000. This paper was recommended by Associate Editor S. Lakshminarayanan.

The authors are with the Department of Information Management, National Sun Yat-sen University, Kaohsiung, Taiwan 804, R.O.C.

Publisher Item Identifier S 1083-4427(00)03706-1.

TABLE I
THE MAPPING OF PROPERTIES BETWEEN
THE MAIS AND SWARM

Multi-Agent Information System	Swarm Simulation System
• Agent	• Object-oriented representation
• Self-organization	• Nested inherent hierarchy
• Autonomous agent behavior	• Individual-based modeling
• Distributed and loosely coupled control	• Parallel and concurrent processing

environment, selects and performs actions to respond to its environment, and after each action, receives feedback from the environment. Such feedback called reinforcement can be positive (reward), negative (punishment), or zero. The positive reinforcement tends to encourage a learner to respond to the stimulus in the future from its environment with the same action, while the negative reinforcement discourages the same response. The feedback from continuous responses to the environment results in an action model to determine which action will be taken under which state.

Reinforcement learning focuses on two problems: temporal credit assignment problem and generalization problem [4]. Temporal credit assignment problem deals with how to assign credit or blame to each individual situation to adjust its decision making and improve its performance after an agent performs a sequence of actions and obtains certain outcomes. Generalization problem concerns how to guess about new situations based on experience with similar situations although the learner does not explore all problem space. Temporal difference (TD) methods are well known approaches to the temporal credit assignment problem [7]. The adaptive heuristic critic (AHC) and Q-learning are two forms of TD-based reinforcement learning [7], [9]. Although several studies found the superiority of Q-learning over AHC-learning, but whether Q-learning works better than AHC-learning in general is still an open question [4]. For the purpose of investigating the combination of simulation and learning in designing new business processes, we adapt Q-learning as the learning agent in Swarm agent development.

In our implementation, we apply QCON architecture [4], which uses a neural network (called the *utility network*) to model the utility function $util(x, a)$. Given a utility network and a state x , the agent's policy is to choose the action for which $util(x, a)$ is maximal. The learning architecture is denoted in Fig. 1, where the utility network is used for learning the utility function by accepting world state and reinforcement values, and the stochastic action selector chooses an action according to utilities. The algorithm for the QCON framework is described as follows.

- Step 1: $U_i \leftarrow util(x, i)$, for each action i , and x is the current state;
- Step 2: $a \leftarrow select(U, T)$;
- Step 3: Perform action a , and obtain new state y and reinforcement r ;
- Step 4: $u' \leftarrow r + \gamma \cdot Max\{util(y, k) | k \in actions\}$;
- Step 5: Adjust utility network through input state x by back-propagating error $\Delta U = \begin{cases} u' - U_i, & \text{if } i = a \\ 0, & \text{otherwise} \end{cases}$;
- Step 6: Go to Step 1, until $\Delta U \leq \varepsilon$ (error acceptance threshold).

The structure of the back-propagation neural network for the utility network uses the state and action parameters as the input nodes, and the utility value as the output nodes.

IV. BUSINESS PROCESS SIMULATION AND LEARNING SYSTEM (BPSLS)

The Swarm simulation system captures the properties of MAIS, such as distributed, autonomous, interdependent, and self-organizing (aforementioned in Table I). We extend Swarm's adaptability by adding the reinforcement learning module into the learning mechanism in an agent. The Swarm simulation system incorporated with

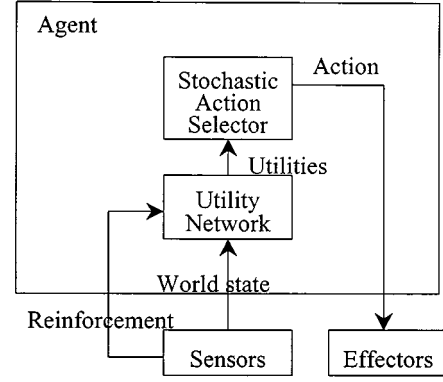


Fig. 1. A Q-learning framework: QCON.

the Q-learning agent (*Q-SwarmObj*) results in a business process simulation and learning system (BPSLS), as shown in Fig. 2. The learning agent performs the following four steps starting from receiving messages and ending with sending out messages.

- Step 1: The learning mechanism in an agent receives the message. The message may come from its parent Swarm or other sibling agents.
- Step 2: The learning mechanism forward the state and incoming message to the Q-learning agent (*Q-SwarmObj*). The learning mechanism works as a learning-task dispatcher and memorizes the learning parameters and utilities returned from *Q-SwarmObj*.
- Step 3: The Q-learning agent (*Q-SwarmObj*) performs the reinforcement learning task, and returns the action utilities.
- Step 4: The action function sends message out to other agents after determining actions according to the action utilities.

V. THE APPLICATION OF THE BPSLS FOR THE OFP IN SCNS

A. The BPSLS System Implementation

We apply the BPSLS for simulating and learning the order fulfillment process (OFP) in supply chain networks (SCN's). The OFP in a SCN is a business process starting from receiving customer orders, and ending with delivering products to the customers. Business partners in the network are autonomous or semi-autonomous organizations, which coordinate each other to fulfill orders according to the organizational roles in the process. Due to the precedent dependence of components for products, partners are interdependent, and communicate through certain information infrastructure, although they may be distributed geographically and organizationally. Therefore, we view the OFP in SCN's as a multi-agent information system, and Swarm is a suitable tool to simulate it.

Fig. 3 illustrates the application of the BPSLS to simulate and learn the OFP in SCN's. The *OFP Observe Swarm* initiates and controls the whole simulation. It creates two swarms: *OFP Model Swarm* and *Statistics Swarm*. *Statistics Swarm* computes the statistics gathered from the simulation for analysis purpose. The *OFP Model Swarm* is composed of an array of *SCN Entities* created while building objects to represent business partners in the SCN. The SCN configuration with each entity's properties and product information are fed in during the entity creation. The *OFP Model* actions are composed of each SCN entity's action, and then activate them when the *OFP Model Swarm* is activated. Each SCN entity is composed of a set of agents, such as order management agent, production planning agent, manufacturing agent, inventory agent, demand policy management agent, and learning agent, in the implementation.

The enclosed agents are described as follows:

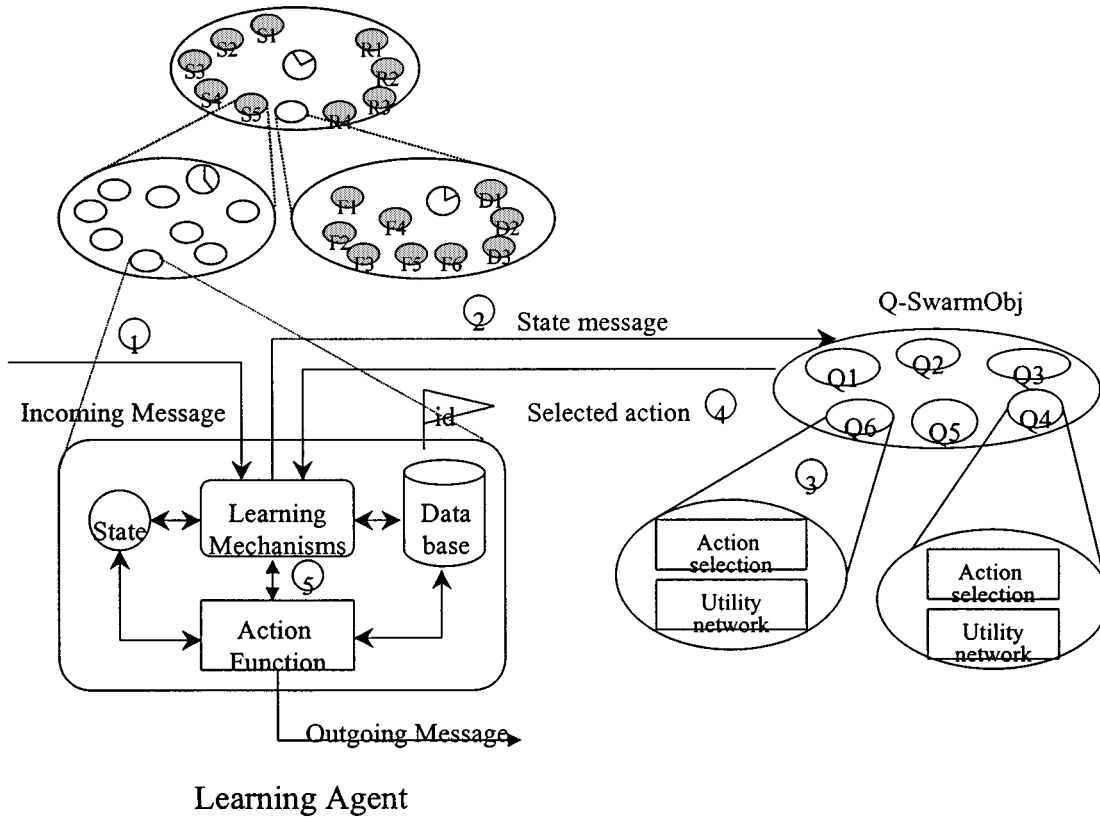


Fig. 2. Business process simulation and learning system (BPSLS).

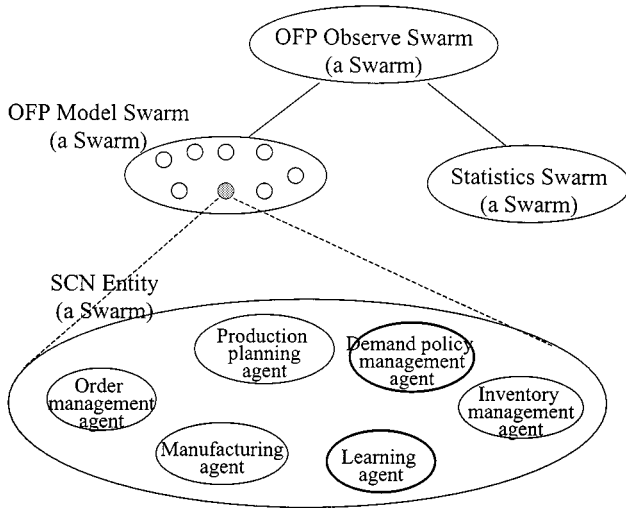


Fig. 3. The BPSLS for simulating and learning the OFP in SCN's.

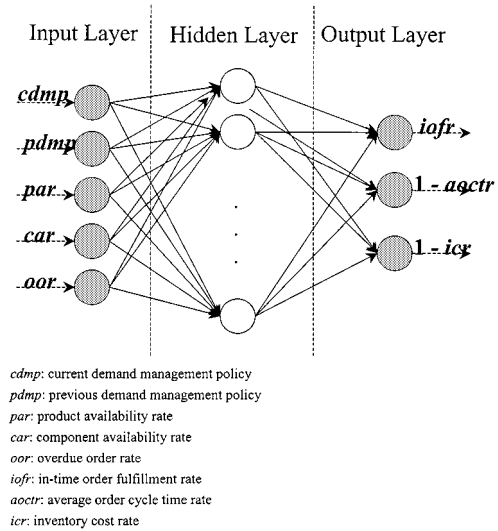


Fig. 4. The three-layer feed-forward neural network as the utility network.

- 1) The order management agent receives and processes orders, and sends to outside entities or other agents inside the entity. Its tasks include estimating due dates, checking inventory or production status, and sending orders to upstream suppliers or to production planning agent.
- 2) The inventory management agent handles tasks related to inventory, such as keeping track of inventory records, determining the reordering point, and responding to inventory status queries from other agents.
- 3) The production planning agent generates build plans according to the demand management policy.

- 4) The manufacturing agent executes the manufacturing process.
- 5) The demand policy management agent is designed to coordinate the tasks performed by different agents to follow certain demand management policy. So that, the entity can switch between different policies through the demand policy management agent.
- 6) The learning agent strengthens the adaptability of the SCN entity by taking the status information from other agents, performing reinforcement learning by sending state information to *Q-SwarmObj*, and then directing the *demand policy management agent* to switch among demand management policies.

TABLE II
OFP PERFORMANCE COMAPRISONS IN A SMALL ORDER QUANTITY ENVIRONMENT (t -TEST RESULTS)

Policy comparison	Statistics	In-time order fulfillment rate	In-time order fulfillment number	Average order cycle time rate	Order cycle time	Inventory cost rate	Inventory cost
Learning(u1)	t-value	3.6041*	2.82312*	-4.36181	0.8166	-32.9028	12.0383*
MTO(u2)	p-value	0.00075*	0.00482*	0.00011	0.21127	3.8E-21	1E-11*
Learning(u1)	t-value	-4.89396	-1.31537	-4.9794	0.60749	-4.59156	-10.2189
ATO(u2)	p-value	3E-05	0.10067	2.5E-05	0.27474	6.4E-05	2.5E-10
Learning(u1)	t-value	-10.3544	-3.26117	-9.46617	2.36405*	10.0376*	-26.644
MTS(u2)	p-value	2E-10	0.00172	1.1E-09	0.01345*	3.6E-10*	4.3E-19

$H_0 : (u1-u2) \leq 0, \alpha=0.05, \text{critical value}=1.71387,$

* denotes significance (reject H_0)

TABLE III
OFP PERFORMANCE COMPARISONS IN A LARGE ORDER QUANTITY ENVIRONMENT (t -TEST RESULTS)

Policy comparison	Statistics	In-time order fulfillment rate	In-time order fulfillment number	Average order cycle time rate	Order cycle time	Inventory cost rate	Inventory cost
Learning(u1)	t-value	26.5381*	26.1313*	10.5072*	-5.75541	-31.0378	13.851*
MTO(u2)	p-value	4.7E-19*	6.6E-19*	1.5E-10*	3.7E-06	1.4E-20	6E-13*
Learning(u1)	t-value	-7.32664	-4.96282	-5.50866	4.34362*	-1.59879	5.77143*
ATO(u2)	p-value	9.4E-08	2.6E-05	6.7E-06	0.00012*	0.06176	3.5E-06*
Learning(u1)	t-value	-14.5742	-12.7935	-10.5402	7.50918*	11.0244*	-16.522
MTS(u2)	p-value	2.1E-13	3E-12	1.4E-10	6.2E-08*	5.9E-11*	1.5E-14

$H_0 : (u1-u2) \leq 0, \alpha=0.05, \text{critical value}=1.71387,$

* denotes significance (reject H_0)

B. Utility Network Structure

The application of the BPSLS to the order fulfillment process requires a set of parameters to specify the structure of neural networks for the QCON architecture, and the attributes for denoting the state and utility values. A three-layer feed-forward neural network trained by the back-propagation method is selected as the basic structure for QCON's utility network as shown in Fig. 4. The input layer consists of six input nodes to input state parameters ($cpdm$, $pdmp$, par , car , cur , and oor), and the output layer consists of three output nodes ($iofr$, $aocr$, and icr). The parameters are defined as below. Note that, we present the reinforcement learning cycle as LCT and the moving average of demand quantity as $MADQ$.

1) State parameters

- Demand management policies (dmp), including the currently ($cdmp$) and previously ($pdmp$) used policies,
- Product availability rate (par), calculated as $par = apq / (LCT \times MADQ)$, where apq denotes the available product quantity in one learning cycle.
- Component availability rate (car), calculated as $car = acq / (LCT \times MADQ)$, where acq denotes the available component quantity in one learning cycle.
- Capacity utilization rate (cur), represented as the utilization rate for finishing the current products in queue and the estimate production in the next learning cycle, which is calculated as $cur = [(time \text{ for finishing products for current waiting queue}) + (LCT \times MADQ) \times time \text{ for producing one product}] / (LCT \times time \text{ ratio used for production in each time unit})$. $cur = [(time \text{ for finishing products for current waiting queue}) + (LCT \times MADQ) \times time \text{ for producing one product}] / (LCT \times time \text{ ratio used for production in each time unit})$.
- Overdue order rate (oor), calculated as $oor = (\text{overdue order quantity}) / (\text{total order quantity} - \text{quantity held in stock})$.

2) Utility parameters

- In-time order fulfillment rate ($iofr$), calculated as $iofr = m/n$, where m denotes the number of orders fulfilled in time during one learning cycle, and n denotes the total incoming order during one learning cycle.
- Average order cycle time rate ($aocr$), calculated as $aocr = (ta_1/te_1 + ta_2/te_2 + 8 + ta_m/te_m) / m$, where m is the total number of orders, ta_i is the actual order cycle time, te_i is the expected order cycle time. We apply $(1 - aocr)$ to the output nodes of the utility network, in the rationale to represent better performance with higher utility value.
- Inventory cost rate (icr), calculated as (actual inventory cost)/(inventory cost for all products during the learning cycle). The inventory cost for all products during the learning cycle is computed by (product quantity) $\times$ (unit inventory cost) $\times LCT$. In the implementation, we use $(1 - icr)$ for the output node.

VI. PERFORMANCE EVALUATION

In this section, we evaluate the BPSLS in order to assess its business process simulation and learning performance. The OFP performance in Swarm is evaluated under three different demand policies: 1) *make-to-order* (MTO), 2) *assembly-to-order* (ATO), and 3) *make-to-stock* (MTS) policies. In the experiments, we identify a proper utility neural network structure with eight hidden nodes and the training rate 0.4. We compare OFP performance between using the fixed policy combination and applying the reinforcement learning to dynamically select policies. In order to obtain more general results, we evaluate the OFP in two kinds of order patterns: *large order quantity* and *small order quantity*. The t -test results are listed in Tables II and III, where we keep the actual performance outcomes in order cycle time, inventory cost, and in-time order fulfillment number, in addition to the in-time order fulfillment rate ($iofr$), average order cycle time rate ($aocr$), and inventory cost rate (icr).

TABLE IV
OFP PERFORMANCE COMPARISONS IN A LARGE ORDER QUANTITY ENVIRONMENT WITH PRE-TRAINED UTILITY NETWORK (*t*-TEST RESULTS)

Policy comparison	Statistics	In-time order fulfillment rate	In-time order fulfillment number	Average order cycle time rate	Order cycle time	Inventory cost rate	Inventory cost
Learning(u1)	t-value	32.152*	30.9365*	11.3149*	-7.01371	-35.2948	13.2933*
MTO(u2)	p-value	6.4E-21*	1.5E-20*	3.5E-11*	1.9E-07	7.8E-22	1.4E-12*
Learning(u1)	t-value	-7.1993	-6.41593	-5.50231	4.63338*	1.96145*	-16.5334
ATO(u2)	p-value	1.2E-07	7.6E-07	6.8E-06	5.8E-05*	0.03102*	1.5E-14
Learning(u1)	t-value	-16.3793	-14.8892	-12.6998	9.16336*	16.116*	-16.9655
MTS(u2)	p-value	1.8E-14	1.3E-13	3.5E-12	1.9E-09*	2.5E-14*	8.4E-15

$H_0 : (u1-u2) \leq 0, \alpha=0.05, \text{critical value}=1.71387,$

* denotes significance (reject H_0)

In a small order quantity environment (as shown in Table III), the dynamic policy assignment using reinforcement learning significantly outperforms the MTO policy in the measurements of in-time order fulfillment rate and in-time order fulfillment number, but significantly increases the inventory cost. The dynamic policy assignment significantly outperforms the MTS policy in the measurements of inventory cost rate, but significantly increase the order cycle time. The dynamic policy assignment does not significantly result in different OFP performance with the ATO policy.

In a large order quantity environment (as shown in Table III), the dynamic policy assignment outperforms the MTO policy in order cycle time rate, which implies that the learning capability increase the customer satisfaction in delivering products in a shorter cycle time. However, the dynamic policy assignment results in longer cycle time and inventory cost comparing with the ATO policy. This inspires us to further investigate how to enhance the dynamic policy assignment.

We train the utility network using a sequence of data (the number of training data records $\gg 5$) collected from the OFP prior to the reinforcement learning, so that the learning agent possesses more experiences before starting the policy assignment. The results show that the dynamic policy assignment significantly outperforms the ATO policy in the inventory cost rate (Table IV). Therefore, we anticipate more improvement will occur if more experience learned by the learning agent.

VII. CONCLUSIONS AND FUTURE RESEARCH

The MAIS incorporated with the learning capacity, called BPSLS, shows its strengths in designing new business processes. The BPSLS enhances the process adaptability by dynamically assigning demand management policies to the order fulfillment process in supply chain networks, in addition to simulate and analyze business processes. Through a sequence of evaluations, we capture the characteristics of the BPSLS in the utility-network structure, training rates, and the contribution of experience to the OFP improvement under different order patterns. However, more efforts need to be spent in the future to investigate the applicability and limitation of the BPSLS. For example, the length of a reinforcement learning cycle, policy switching cost, the parameter value range, and various business processes in addition to the OFP.

REFERENCES

- [1] T. H. Davenport, *Process Innovation: Reengineering Work Through Information Technology*. Cambridge, MA: Harvard Business School Press, 1993.
- [2] M. Hammer and J. Champy, *Reengineering the Corporation: A Manifesto for Business Revolution*. New York: Harper Business, 1993.
- [3] F.-R. Lin, "Reengineering their order fulfillment process: A multi-agent information system approach," Ph.D. dissertation, University of Illinois at Urbana-Champaign, 1996.

- [4] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Mach. Learn.*, vol. 8, no. 3-4, pp. 293-321, 1992.
- [5] W. J. Kettinger, S. Guha, and J. T. C. Teng, "Business process change: A study of methodologies, techniques, and tools," *MIS Quart.*, pp. 55-80, March 1997.
- [6] Swarm Web Page: <http://www.swarm.org>
- [7] R. S. Sutton, "Learning to predict by the methods of temporal difference," *Mach. Learn.*, vol. 3, pp. 9-44, 1988.
- [8] D. G. Wastell, P. White, and P. Kawalek, "A methodology for business process redesign: Experiences and issues," Information Process Group, Department of Computer Science, University of Manchester, U.K., Technical Report, 1996.
- [9] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3-4, pp. 279-292, 1992.

Hybridization of Gradient Descent Algorithms with Dynamic Tunneling Methods for Global Optimization

Pinaki RoyChowdhury, Y. P. Singh, and R. A. Chansarkar

Abstract—An algorithm based on gradient descent techniques with dynamic tunneling methods for global optimization is proposed. The proposed algorithm consists of gradient descent for local search and a direct search scheme, based on dynamic tunneling technique, for repelling away from local minimum to find the point of next local descent. This search process applied repeatedly finds the global minimum of an objective function. The convergence properties of the proposed algorithm is validated experimentally on benchmark problems. A comparative computational results confirm the importance of dynamic tunneling in gradient descent techniques.

Index Terms—Dynamic tunneling, global optimization, Lipschitz condition.

I. INTRODUCTION

Global optimization is the task of finding the absolutely best set of parameters to optimize an objective function [1], [2]. In general, there can be solutions that are locally optimal but not globally optimal [3]. Consequently, global optimization problems are typically quite difficult to solve exactly and they are often NP-hard [4]. It is also well

Manuscript received January 8, 1998; revised January 19, 2000. This paper was recommended by Associate Editor P. Hansen.

P. RoyChowdhury and R. A. Chansarkar are with Defence Terrain Research Laboratory, Defence Research And Development Organization, Delhi, India. Y. P. Singh is with Faculty of Information Technology, Multimedia University, Selangor, Malaysia.

Publisher Item Identifier S 1083-4427(00)03427-5.